HARP: ExactFrac-Constrained Score Serving for GNN Prediction APIs

Krish Sharma
Academy of Math, Science, and Applied Technology
Ed W. Clark High School
& Data Science Laboratory (DiSC), UNLV
Las Vegas, NV, USA
krish.sharma@unlv.edu

Shaikh Arifuzzaman
Data Science Laboratory (DiSC)
Department of Computer Science
University of Nevada, Las Vegas
Las Vegas, NV, USA
shaikh.arifuzzaman@unlv.edu

***Abstract*—High-volume GNN prediction APIs serve softmax scores for ranking, routing, A/B tests, and compliance pipelines on graphs from thousands to millions of nodes; production stacks cache by request hash and re-query the same entities under continuous traffic. Operators sometimes also need a positive fraction of *raw* model posteriors—unmodified $\hat{p}_v$ for calibration and downstream score consumers—which is stricter than replaying a previously sampled noisy response from a cache or ledger. Fresh uniform release noise cannot meet that raw-exact SLA; global caches and seeded noise raise ReplayFrac without restoring ExactFrac≡RawExactFrac. HARP is a Big-Data score-serving layer for this constrained setting: constructors rank nodes, hop expansion forms a protected set, a protector runs only there, and validation-time Frac search targets Acc under ExactFrac$\geq c$ (study default c=0.60) with an optional audit dial τ. On defense-aware LiRA (n_{shadows}=16), HARP is feasible for c>0 with competitive Acc/ECE on large-graph serving probes; residual clean-slice risk is reported as a structural limit of ExactFrac> 0.**

***Index Terms*—big data prediction APIs, graph neural networks, response caching, ExactFrac, selective score serving, membership audit**

I. Introduction

A. Motivation

GNN prediction APIs return softmax vectors for triage, ranking, and analytics at Big-Data scale [9], [10], [28]: graphs with 10^3–10^6+ nodes, high query velocity (clients re-query the same entities), and veracity requirements for downstream score consumers. Production serving stacks hash requests and cache responses to cut latency under repeated queries [40], [41]. Released scores also leak training membership [1], [2], [4], [5]. This paper is about the *systems* intersection of those facts: how to release scores under a raw-posterior service contract while still spending noise where an audit requires it—not a new membership-inference theory paper.

a) Why ExactFrac is operationally necessary.: Two contracts are easy to confuse. *ReplayFrac* asks whether two queries return the same vector (a cache hit or sticky/seeded draw can satisfy this with a *noisy* first answer). *ExactFrac*≡RawExactFrac asks whether the released vector equals the unmodified model posterior $\hat{p}_v$. Raw equality matters when downstream systems treat API scores as model truth: temperature calibration, threshold routers that must match offline evaluation, A/B analytics that join on score bits, and compliance pipelines that attest "this is the model's output." Caching a noisy draw restores ReplayFrac and QPS; it does *not* restore ExactFrac. We therefore study

$$\max_{\Pi} \text{Acc}(\Pi) \quad \text{s.t.} \quad \text{ExactFrac}(\Pi) \geq c, \quad \text{LiRA}(\Pi) \leq \tau$$

(optional τ), with study default c=0.60 as an operator dial (Fig. 2). Fresh uniform LBP has both fractions ≈0 (Remark 1). Table VIII shows global-cache and seeded LBP achieve ReplayFrac=1 with ExactFrac=0; selective release (HARP) meets c>0 without a ledger of non-raw scores.

b) How to read Acc / ExactFrac / LiRA.: **Acc** is task utility (primary objective under the SLA). **ExactFrac** is the service constraint: fraction of nodes released as raw $\hat{p}_v$. **LiRA** is an optional audit dial—used to decide how much Frac to spend, not the headline win condition. In practice: if ExactFrac$\geq c$>0 is required, discard rows with ExactFrac=0 even if their LiRA looks better; if only Acc s.t. LiRA$\leq \tau$ matters, prefer equal-mass LBP and ignore HARP. Our running Cora example (c=0.60, n_{sh}=16): equal-mass LBP Acc≈0.87/LiRA≈0.59/ExactFrac=0 vs. HARP Acc≈0.83/LiRA≈0.67/ExactFrac=0.60 with much lower clean-majority ECE.

c) Framework.: **HARP** is a constrained score-serving layer for GNN APIs: (1) a *constructor* ranks nodes (topology, degree, train-neighbor exposure, confidence/entropy, random, or audit-guided vulnerability); (2) hop expansion $\mathcal{P}_k = \text{Ball}_k(\mathcal{R})$ plus a *protector* only on $\mathcal{P}_k$; (3) operator dials Frac/Mass/session budget B, with **CFS** searching Frac under (c, τ). Default: *release-only* selective Laplace at Frac=0.40 (ExactFrac=0.60).

d) Headline evidence.: At n_{shadows}=16, HARP meets ExactFrac$\geq$ 0.60 while uniform and stateful-noisy LBP cannot; equal-mass LBP remains the right comparator when c=0 is allowed. Within the constrained class, constructors cut clean-slice AUROC (0.669 → 0.594), CFS selects Frac under operator (c, τ), and HARP∘GAP preserves GAP's ε for public-metadata constructors. ogbn-arxiv/products probes show Acc and serving cost at volume; privacy audits at that scale are scoped and canary-stressed (not claimed as full-graph LiRA).

B. Gap

Prior release defenses (LBP [5], MemGuard [6]) perturb every response and therefore yield ExactFrac=0 unless wrapped in a response ledger that stores non-raw scores. Train-time GNN MIA methods (GTD, MaskArmor) [21], [22] and DP-GNNs [13], [14], [36] do not expose ExactFrac/Mass serving accounting for prediction APIs. None formalize Acc under ExactFrac$\geq c{>}0$ as a Big-Data score-serving objective with constructors, validation-time Frac search, and trainer composition.

C. Contributions

1) **ExactFrac-constrained API release.** Separate RawExactFrac from ReplayFrac for prediction serving; show fresh and stateful-noisy uniform policies fail ExactFrac$\geq c{>}0$; hop coverage and DP post-processing composition (Prop. 2).
2) **HARP serving layer.** Selective, hop-aware release with constructors, validation-time CFS, optional DCS, and HARP$\circ$GAP for finite ε plus ExactFrac controls under continuous query traffic.
3) **Constraint-aware Big-Data evaluation.** Headline LiRA at $n_{\mathrm{sh}}{=}16$; RawExactFrac-vs-ReplayFrac SLA (Table VIII); cache microbenchmarks; scoped OGB Acc/serving and canary-stressed probes; clean-slice risk as a structural limit.

II. Background and Related Work

Shadow/confidence MIAs [1]–[3] and LiRA [4] are the audit standard for prediction APIs; overconfidence is a recurring cue [17]. On graphs, Olatunji et al. [5] introduce node-level MIAs and LBP; He et al. [38], GTD [21], MaskArmor [22], and surveys [8] study related attacks/defenses. MemGuard [6] adversarially perturbs every confidence vector (ExactFrac=0). DP-GNNs such as GAP [13], [14] provide train-time (ε, δ); structure modulates leakage [18], [39]. **Positioning:** This is a prediction-API / score-serving paper with an optional membership audit dial. LBP/MemGuard spend on every node (ExactFrac=0 unless a non-raw ledger is added); GTD/MaskArmor harden training without ExactFrac accounting; GAP answers formal ε (ExactFrac=1 at eval for train-only noise); HARP answers ExactFrac$\geq c$ serving under continuous query traffic and composes with DP trainers under Prop. 2. Our GAP-agg and MemGuard baselines are faithful reimplementations, not line-by-line ports of the official codebases—we label them as such throughout. Primary comparisons under $c{>}0$ are selective policies at matched Frac; equal-mass LBP is the unconstrained ($c{=}0$) reference.

III. Problem Setup

A. Notation

Let $G = (V, E)$ be an undirected graph with $n = |V|$ nodes, feature matrix $X \in \mathbb{R}^{n\times d}$, and labels $y \in \{0, \ldots, C-1\}^n$. A random split partitions V into supervised train $\mathcal{T}$, validation $\mathcal{V}$, and test $\mathcal{Q}$ with ratios $40\%/20\%/40\%$ (resampled per seed). Training is *transductive*: every node participates in message passing; only $\mathcal{T}$ enters the cross-entropy loss. We write $\hat{p}_v$ for the softmax posterior at v, r_v for a risk score in $[0, 1]$, $\mathcal{P}_k$ for the hop-expanded protected set, Mass$= \sum_v \sigma_v$, and Frac$= |\mathcal{P}_k|/n$.

B. Constrained score release

Operators rarely optimize Acc alone. They may need usable accuracy *and* a positive fraction of unmodified model posteriors, subject to a membership audit.

Definition 1 (Constrained Score Release)**.** *Given a trained GNN and release channel, choose a release policy* Π *to*

$$\max_{\Pi} \ \mathrm{Acc}(\Pi) \quad \textit{s.t.} \quad \mathrm{LiRA}(\Pi) \leq \tau, \quad \mathrm{ExactFrac}(\Pi) \geq c,$$

where $\mathrm{ExactFrac} \equiv \mathrm{RawExactFrac} = \Pr_v[\tilde{p}_v = \hat{p}_v]$ *(fraction of nodes released as the unmodified posterior),* $\mathrm{ReplayFrac} = \Pr_v[\tilde{p}_v^{(1)} = \tilde{p}_v^{(2)}]$ *is reported separately,* τ *is an* operator-chosen *audit risk tolerance matched to the LiRA budget (not a universal privacy target), and* $c \in [0, 1]$ *is an operator-chosen clean fraction (study default* 0.60*). Under release-only selective Laplace with weak*$=0$*, ExactFrac*$=1-\mathrm{Frac}$ *by construction and implies ReplayFrac on the clean slice without a response ledger.*

Remark 1 (Uniform and stateful-noisy LBP under $c{>}0$)**.** *Independent Laplace noise with scale* $b{>}0$ *on every coordinate changes* $\hat{p}_v$ *almost surely, so ExactFrac*$= 0$ *for any uniform fresh LBP policy. A global per-node response cache or a deterministic PRNG keyed by (node, epoch) can raise ReplayFrac to* 1 *and restore cache hits / ledger audit, but ExactFrac remains* 0 *(Table VIII). Selective release with protected fraction Frac achieves ExactFrac*$= 1-\mathrm{Frac}$ *by construction, without storing non-raw scores.*

This reframes the Acc–LiRA comparison. Equal-mass LBP can win Acc subject to LiRA$\leq \tau$ when $c{=}0$ is allowed; HARP solves the strictly harder problem with ExactFrac$\geq c{>}0$. The Frac sweep traces HARP's Pareto frontier under that constraint.

C. Threat model

The adversary is a black-box client: it queries softmax scores and tries to guess whether a node was in the training set [2], [4], [5]. It does not see gradients or embeddings. White-box parameter access remains out of scope; label-only gap AUROC and edge-similarity membership AUROC are reported alongside LiRA (GAP-agg cuts edge AUROC $0.895 \to 0.748$ on Cora).

We also stress three practical variants: defense-aware shadows, multi-query averaging ($K \in \{1, 5, 20\}$), and an LTE-aware query adversary that focuses on clean nodes, protected-set boundaries, and top-decile LTE seeds (Sec. VI-B0l). We do not claim robustness to poisoning or unbounded queries without a session budget.

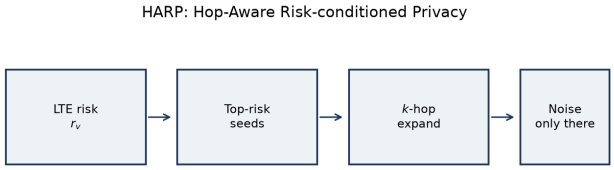

Fig. 1. HARP pipeline: risk ranking → high-risk seeds → k-hop expansion → protector only on the protected set.

D. Metrics

Utility: test Acc (primary). **Audit:** LiRA AUROC [4] and TPR@1% FPR, plus Salem-style ϕ/confidence attacks [2]. **Fidelity:** ECE [17], clean-slice ECE, ExactFrac. LiRA decides *whether* to spend noise; Acc/ECE measure the cost.

Remark 2. *The primary protocol is transductive (non-members remain in the graph during training), matching common semi-supervised GNN APIs. We additionally report an inductive edge-exclusion check on Cora (Sec. VI-B0m) where test-incident edges are dropped at train/infer time.*

IV. HARP: Hop-Aware Selective Release

A. Design principle

Uniform *fresh* posterior noise cannot meet ExactFrac$\geq c{>}0$—any independent $b{>}0$ draw changes $\hat{p}_v$. Stateful wrappers (global caches, ledgers, seeded PRNGs) can meet ReplayFrac without ExactFrac (Table VIII). Node-local selection without hop coverage is incomplete for GNNs: if v is protected but neighbor u is released exact, aggregation can reintroduce membership signal into u's score. HARP is a three-interface serving framework (Fig. 1):

1) **Constructor (pluggable):** rank nodes—topology (LTE), degree, train-neighbor exposure, confidence/entropy, random, or audit-guided vulnerability.
2) **Geometry + protector:** expand $\mathcal{P}_k = \mathrm{Ball}_k(\mathcal{R})$; apply Laplace or masking only on $\mathcal{P}_k$.
3) **CFS + dials:** validation-time search of Frac under ExactFrac$\geq c$ and optional LiRA$\leq \tau$; log Mass/Frac/B.

Locked study config: *release-only* Laplace, Frac=0.40 (ExactFrac=0.60), k=1, σ=0.30. Topology LTE is the no-shadow default constructor; ensemble/audit-guided ranking is preferred when shadows are already paid for. Train-time alignment is an optional ablation—not required for the ExactFrac claim.

a) Noise mass and clean fraction.: $\mathrm{Mass} = \sum_v \sigma_v$ and $\mathrm{Frac} = |\mathcal{P}_k|/n$ are operator accounting metrics. Uniform LBP at scale b has Mass=bn and Frac=1. See paragraph IV-E0a for the locked-budget identity.

b) Locked-config example (Cora).: On Cora (n=2708), Frac=0.40 protects ≈1083 nodes after one-hop expansion; ≈1625 nodes stay exact. $\mathrm{Mass}_{\mathrm{HARP}}$≈325 matches equal-mass LBP with b=0.12. Strong LBP (b=0.3) spends Mass≈812 on all nodes—the Acc-collapse baseline in Table II.

c) Release transform (Laplace protector).: Let $\sigma_v = \sigma_{\mathrm{strong}}\mathbb{1}[v \in \mathcal{P}_k]$. The released posterior is

$$\tilde{p}_v = \mathrm{renorm}\big([\hat{p}_v + \mathrm{Lap}(0, \sigma_v)]_+\big), \tag{1}$$

with independent Laplace coordinates and the same transform on every defense-aware shadow. The masking protector instead returns $\hat{p}_v$ for $v \notin \mathcal{P}_k$ and a one-hot of $\arg\max \hat{p}_v$ for $v \in \mathcal{P}_k$.

d) Algorithm (systems view).:

1) **Risk:** $r \leftarrow \mathrm{Constructor}(G, \mathcal{T})$ (topology / degree / confidence / random / audit).
2) **CFS (optional):** search Frac$\leq 1-c$ to maximize Acc subject to LiRA$\leq \tau$.
3) **Seeds + expand:** binary-search seeds so $|\mathrm{Ball}_k(\mathcal{R})|/n \approx$ Frac; $\mathcal{P}_k \leftarrow \mathrm{Ball}_k(\mathcal{R})$.
4) **Release:** apply (1) (or masking); log $(\mathrm{Mass}, \mathrm{Frac}, \mathrm{Acc}, \mathrm{LiRA}, \mathrm{ECE})$.

Train-time alignment is optional and omitted in the locked release-only configuration.

B. Risk constructors

The constructor interface is part of the framework contribution. **Topology (LTE)** uses only G, $y|_{\mathcal{T}}$, and the train mask (inverse degree, local heterophily, train-neighbor exposure); it is a cheap default when shadows are unavailable—not claimed to be privacy-optimal. **Degree / train-neighbor** isolate individual topology cues. **Confidence / entropy** rank from a clean forward pass (high max-confidence or high entropy). **Random** is the matched-Frac selectivity control. **Audit-guided** scores $|\mu_{\mathrm{IN}} - \mu_{\mathrm{OUT}}|$ over a small shadow ensemble [37] and is preferred when that ensemble is already paid for. We report Spearman correlation between each constructor and audit vulnerability, plus Acc/LiRA/slice AUROC at matched Frac (Table VII). On Cora/Citeseer, *entropy* is the strongest cheap prior (Spearman ≈0.33–0.35); topology (LTE) is moderate (≈0.25–0.28); max-confidence is anti-correlated. Heterophilic Chameleon priors are weak—selectivity still enforces ExactFrac, but ranking gains are limited. Where topology underperforms random on LiRA (e.g., Cora in Table VII), we state that plainly: ExactFrac comes from selectivity, not from LTE.

C. Why k-hop expansion

Let $\mathcal{R}$ be seeds. A GNN score at u already mixes information from $\mathrm{Ball}_k(u)$ before any release noise is applied. If a membership-sensitive seed $v \in \mathcal{R}$ is noised at release but a neighbor u is returned bit-exact, the adversary simply queries u.

Proposition 1 (Hop coverage under a one-hop linear mixer)**.** *Fix a one-hop score model $s_u = \alpha\, z_v + (1-\alpha)\, z_u$ for $u \in \mathcal{N}(v)$, $\alpha \in (0, 1]$, where z_v carries a membership cue of gap $\gamma > 0$, and release adds independent Laplace noise only to chosen nodes' scores. If the policy noises v but leaves u exact, an adversary observing s_u retains the unprotected leaf advantage. Noising $\mathrm{Ball}_1(\{v\})$ places Laplace noise on every score that linearly depends on z_v under this mixer.*

Proof sketch. Under seed-only protection the released leaf equals the clean mixer output, so the member/non-member gap in $\mathbb{E}[s_u]$ remains $\alpha\gamma$. Expanding protection to u subjects that coordinate to $\mathrm{Lap}(0,\sigma)$. A synthetic mixer confirms the ordering: unprotected and seed-only leaf AUROCs stay near 1.0, while hop-1 protection lowers the leaf attack to ≈ 0.72. □

This is a motivating linearized argument for hop expansion, not a necessity theorem for nonlinear multi-layer GNNs. Empirically we set k to the model's receptive field (default $k{=}1$ for 2-layer SAGE/GCN; $k{\in}\{0,2\}$ changes Acc by $\lesssim 0.01$ at matched Frac). Frac—not k—is the population Acc–Mass knob.

D. Guarantees: CFS configuration and DP composition

Remark 3 (CFS is validation-time configuration, not a novel optimum)**.** *Constrained Frac Search (CFS) enumerates a finite grid $\mathcal{F} \subseteq [0, 1-c]$ and returns $f^\star = \arg\max\{\mathrm{Acc}(f_i) : \mathrm{LiRA}(f_i) \le \tau\}$ using* validation *Acc and a validation-time defense-aware LiRA estimate; the locked tables then report a held-out test Acc / headline LiRA once. This is a reproducible operator procedure for choosing Frac under explicit (c,τ)—not a claim of global optimality over all release policies or constructors. The audit cap τ is an operator risk tolerance tied to the attack budget (study headline: $n_{\mathrm{sh}}{=}16$, $\tau{=}0.70$); $\tau{=}0.55$ is infeasible at that budget, and we report neighboring caps $\{0.68, 0.70, 0.75\}$ (Sec. VI-B).*

Proposition 2 (Post-processing composition)**.** *Let T be any training procedure whose released model satisfies (ε,δ)-DP w.r.t. the training data, and let Π be a HARP release policy that is a (possibly randomized) function only of the model's released outputs and* public *deployment metadata. Then HARP$\circ T$ satisfies the same (ε,δ)-DP (standard post-processing [27]) and attains ExactFrac$=1-$ Frac w.r.t. the DP model's posteriors.*

Proof. Immediate from the post-processing property whenever Π does not re-read private training labels or private membership bits beyond what T already released. □

Prop. 2 is not a new DP theorem; it is the reason HARP can sit *above* a DP trainer as a serving layer. **Constructor caveat:** topology LTE uses the train mask and $y|_{\mathcal{T}}$ as server-side deployment metadata. If those bits are private under the DP threat model, use a public-metadata constructor (degree, confidence/entropy from the released DP outputs, random, or audit scores computed only from released shadow outputs) before claiming Prop. 2. Section VI-B measures HARP$\circ$GAP under the public-metadata reading.

E. Complexity and accounting

Proposition 3 (HARP setup and release complexity)**.** *Let $G=(V,E)$ have $n=|V|$ nodes and $m=|E|$ adjacency endpoints, with C classes. Fix hop radius $k=O(1)$ and $T=O(1)$ binary-search iterations for a target protected fraction. Then setup (LTE + seeds + k-hop expand) runs in $O(m+n\log n)$ time and $O(n+m)$ space. Release runs in $O(nC)$—matching uniform LBP's tier, with less noise-sampling work whenever* $\mathrm{Frac} < 1$.

Proof sketch. LTE scans edges then normalizes n scores: $O(m{+}n)$. Seed selection is $O(n\log n)$ plus T BFS expansions at $O(m+n)$. Release writes scales, samples Laplace on $\mathcal{P}_k$, and renormalizes: $O(nC)$. □

Remark 4 (Per-response Laplace; no global DP claim)**.** *Interpret each node's clean posterior as a query with ℓ_1-sensitivity at most $\Delta{=}2$ on the simplex. Independent Lap(σ) on coordinates of $v \in \mathcal{P}_k$ is a per-response Laplace mechanism with scale σ; unprotected nodes receive no noise. Any policy with ExactFrac>0 has infinite* global (ε,δ)*-DP by construction. We report this only as an accounting remark—not as a local-DP guarantee under graph adjacency—and recommend training-time DP (e.g., GAP) when a legal global ε is required [13].*

a) Accounting identity (stated once).: Under locked Frac and fixed σ_{strong}, $\mathrm{Mass}_{\mathrm{HARP}} = \mathrm{Frac}\cdot n\cdot\sigma_{\mathrm{strong}}$ by construction. Mass is an operational dial correlated with mean ℓ_1 distortion and top-1 flip rate (at matched Mass 325 on Cora: HARP mean ℓ_1 0.349 vs. LBP 0.493); it is not a standard privacy budget. Empirical claims are (i) feasibility under ExactFrac$\ge c$, (ii) Acc/ECE within the selective class at matched Frac/Mass, and (iii) serving telemetry.

F. What HARP is not

HARP is not differential privacy and is not a claim of lower population LiRA than uniform noise under ExactFrac$=0$. It is not a claim that randomization "breaks caching"—stateful caches and seeded noise can stabilize non-raw responses. It is a constrained ExactFrac serving framework: spend a protector on a hop-expanded subset, keep a raw-posterior majority, configure Frac with CFS, and audit residual risk—including on unprotected nodes.

V. Experimental Protocol

A. Pre-declared rules

Primary objective: Acc under ExactFrac$\ge c$ (study default $c{=}0.60$; swept in Fig. 2). **Primary metrics:** Acc, ExactFrac, ECE (fidelity). **Audit metric:** LiRA AUROC (and clean-slice confidence AUROC) as deployment constraints—not the headline win condition. **Operational metrics:** Mass, Frac, ExactFrac (RawExactFrac), ReplayFrac, shared-cache hit rate (Table VIII). Hyperparameters are not selected to minimize test LiRA. Under $c{>}0$, primary baselines are selective policies at matched Frac; equal-mass LBP is the unconstrained ($c{=}0$) reference; sticky/global-cache/seeded LBP are ReplayFrac-positive but ExactFrac-infeasible controls.

B. Systems metrics (definitions)

- **ExactFrac** (≡RawExactFrac): fraction of nodes with $\tilde{p}_v = \hat{p}_v$.
- **ReplayFrac**: fraction of nodes with equal responses under re-query (may use cache/seed state).

- **Mass** $= \sum_v \sigma_v$: operational Laplace-scale spend (correlated with distortion).
- **Frac** $= |\mathcal{P}_k|/n$: protected fraction (ExactFrac=1 − Frac under weak=0).
- **ECE**: calibration of released probabilities.

C. Locked HARP (release-only)

Release-only selective Laplace: λ=0, no HCAG, no train-on-protected; k=1, σ_{strong}=0.30, Frac=0.40 (ExactFrac=0.60). Constructor default in locked tables: topology (LTE); audit/random/degree/confidence appear in Table VII. Train-time alignment is reported only as an ablation. Baselines: equal-mass LBP b=0.12; strong LBP b=0.3; MemGuard; GTD; MaskArmor; selective masking; DP-SGD Acc-tuned (vacuous ε).

D. Baselines and attacks

Gaussian LiRA with defense-aware shadows: n_{shadows}=16 for the headline Cora table and CFS (Tables II, V; τ=0.70), n_{shadows}=4 for multi-dataset grids and Acc dials, and sweeps $\{4, 16, 64\}$ on Cora/Chameleon. Five seeds with mean±95% CI on primary Cora/Chameleon/Citeseer constructor tables; paired seeds across defenses. ogbn-arxiv: NeighborLoader serving Acc with LiRA at $n_{\text{shadows}} \in \{2, 4, 16\}$ (audit stress, not a million-node privacy claim). ogbn-products: full-graph serving RSS/QPS; BFS LiRA ladder at $n \in \{15\text{k}, 40\text{k}\}$, $n_{\text{sh}} \in \{2, 4, 8\}$, plus a canary-stressed 15k probe with undefended canary-LiRA ≈1.0.

E. What each baseline tests

Equal-mass LBP (b=0.12) matches HARP Mass at Frac=0.40 and is the unconstrained (c=0) reference. **Selective constructors** (random/degree/confidence/entropy/audit/topology) isolate ranking quality at matched Frac under the SLA. **Strong LBP** (b=0.3) is a secondary published recipe that collapses Acc under full-graph strong noise. **MemGuard** (reimplementation) fits a standardized MLP ϕ-attack on holdout members/non-members and searches an L_1 ball to drive $P(\text{member}) \to 0.5$ while preserving argmax (ExactFrac=0). Following Aerni et al. [37] we report both ϕ and LiRA; at n_{sh}=16 our port cuts LiRA vs. undefended (0.813 → 0.667) while remaining ExactFrac-infeasible. **GAP-agg** is a GAP-inspired aggregation-Laplace baseline: noise is train-only, so eval ExactFrac=1 with analytical (ε, δ) (hybrid DP-train + ExactFrac-serve). Not a full GAP codebase reproduction. **Selective masking** is an alternate protector (ExactFrac preserved; often worse LiRA). **HARP-full** (Frac=1) is the unconstrained ablation that recovers strong-LBP Acc–LiRA. **GTD/MaskArmor** are train-time defenses (no release Mass/Frac). **Clip+noise DP-SGD** remains an Acc-tuned vacuous-ε reference.

F. Attack protocol (summary)

We train defense-aware shadow models, release transformed posteriors, and report LiRA AUROC with ϕ-features [2], [4].

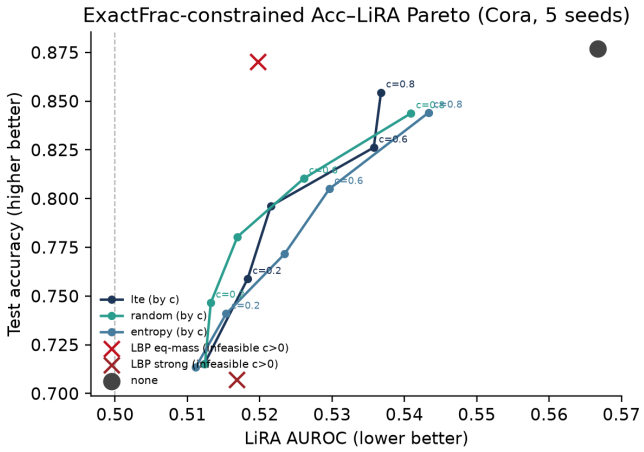

Fig. 2. ExactFrac-constrained Acc–LiRA Pareto on Cora (5 seeds). Curves: selective HARP constructors at $c \in \{0.2, 0.4, 0.6, 0.8\}$ (ExactFrac=c). Uniform LBP markers are ExactFrac=0 and infeasible for every c>0.

Validation nodes are excluded from membership labels; test Acc/ECE use the held-out test split. Multi-query experiments average $K \in \{1, 5, 20\}$ independent draws; session policies cap fresh draws at B=5.

a) Negative controls.: PubMed (near-chance undefended LiRA): HARP still preserves Acc vs. strong LBP at Frac=0.40 (0.839 vs. 0.768); ogbn-arxiv/products give near-chance or memory-bounded audits under matched budgets—audit first, harden second [28].

VI. Results

A. Overview

a) Operational headline (constraint-first).: Under ExactFrac≥ 0.60: (i) HARP is feasible and uniform/stateful-noisy LBP are not (Table I); (ii) release-only HARP keeps Acc competitive within the selective class while returning a 60% *raw*-posterior majority; (iii) cache microbenchmarks show hit-rate gains when many responses are bit-stable (Table IX); (iv) equal-mass LBP can still win Acc–LiRA when c=0 is allowed—that unconstrained privacy-only regime is out of scope for the SLA.

b) Reading the tables.: Compare policies only inside the feasible ExactFrac set for your c. Acc is the primary utility number; LiRA/TPR@1% FPR tell the operator whether to raise Frac or tighten τ; ECE reports score quality on the released (especially clean) majority. Questions: **(Q1)** feasibility under c>0; **(Q2)** Acc/ECE vs. selective and equal-mass baselines; **(Q3)** constructors vs. random on utility *and* slice privacy; **(Q4)** serving cost and clean-slice residual risk; **(Q5)** OGB volume Acc/serving with scoped audits.

B. HARP under ExactFrac≥ 0.60: accuracy and score quality

a) Primary claim.: Fig. 3, Fig. 4, and Table II: under ExactFrac≥ 0.60, release-only HARP is feasible; equal-mass LBP and MemGuard are not. At n_{sh}=16 the undefended attack is strong (LiRA 0.813, TPR@1% 0.200); HARP cuts it to

TABLE I
FEASIBILITY UNDER EXACTFRAC$\geq c$ (RAWEXACTFRAC). STICKY/GLOBAL/SEEDED LBP CAN RAISE REPLAYFRAC BUT REMAIN EXACTFRAC-INFEASIBLE FOR $c>0$.

Policy	c=0	c=0.2	c=0.6	c=0.8
none	yes	yes	yes	yes
LBP fresh	yes	no	no	no
LBP sticky / global / seeded	yes*	no†	no†	no†
HARP (Frac=$1-c$)	yes	yes	yes	yes

*ReplayFrac can be 1 (Table VIII). †ExactFrac=0: responses are never the unmodified $\hat{p}_v$.

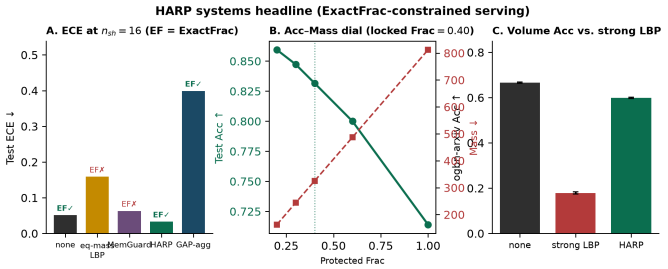

Fig. 3. Systems headline. (A) ECE at n_{sh}=16 with ExactFrac feasibility marks. (B) Frac Acc–Mass dial. (C) ogbn-arxiv Acc vs. strong LBP.

TABLE II
CORA GRAPHSAGE UNDER EXACTFRAC$\geq c$=0.60 (5 SEEDS; LIRA n_{sh}=16). FEASIBLE UNDER $c>0$: NONE, GAP-AGG, HARP. †GAP-INSPIRED AGG. LAPLACE (ANALYTICAL ε; NOT FULL GAP CODE). §SHADOW-FIT MEMGUARD; ϕ = ATTACK IT OPTIMIZES. ¶HARP FRAC=1 RECOVERS STRONG-LBP ACC–LIRA.

Defense	**Feas.**	**Acc**	**LiRA**	**TPR@1%**	ϕ	**ECE↓**	**EF**
none	yes	0.877	0.813	0.200	0.650	0.051	1.00
GAP-agg (σ=3, $\varepsilon\approx$41)†	yes	0.776	0.624	0.039	0.540	0.399	1.00
HARP (release)	yes	0.826	0.674	0.060	0.626	**0.033**	0.60
LBP equal-mass	no	0.870	0.592	0.009	0.584	0.159	0.00
MemGuard§	no	0.877	0.667	0.080	0.619	0.063	0.00
LBP b=0.3 / HARP-full¶	no	0.707 / 0.715	0.551 / 0.560	0.007 / 0.008	0.565 / 0.572	0.142 / 0.152	0.00

0.674 / 0.060 while keeping Acc 0.826, ECE 0.033, and a 60% replay-stable majority. MemGuard is a credible unconstrained baseline (LiRA 0.667) but ExactFrac=0; equal-mass LBP reaches LiRA 0.592 with ECE 0.159 and ExactFrac=0. Equal-mass can win Acc–LiRA when the SLA is dropped—expected and outside the constrained objective.

b) Secondary: Acc vs. published strong LBP.: Against LBP b=0.3 [5], HARP raises Acc 0.707 → 0.832 at Frac=0.40 because strong noise applies to a minority of nodes (Table II). This is mechanically expected and is not the constrained-feasibility claim.

c) Membership audit (deployment dial).: At matched Mass, equal-mass LBP beats HARP on Acc–LiRA (Cora n_{sh}=16: Acc 0.870 / LiRA 0.592 vs. 0.826 / 0.674). If ExactFrac is not required, prefer equal-mass LBP. Under the SLA, LiRA guides whether Frac must rise and whether the audit threshold τ is met after CFS.

HARP Mass 325; LBP eq-mass 325; strong LBP 812. MemGuard is a credible unconstrained baseline (LiRA 0.813 → 0.667) but ExactFrac-infeasible; HARP is within ≈0.01 LiRA of MemGuard *with* ExactFrac=0.60 and much lower ECE. At n_{sh}=4: GTD 0.878/0.538; MaskArmor 0.877/0.580.

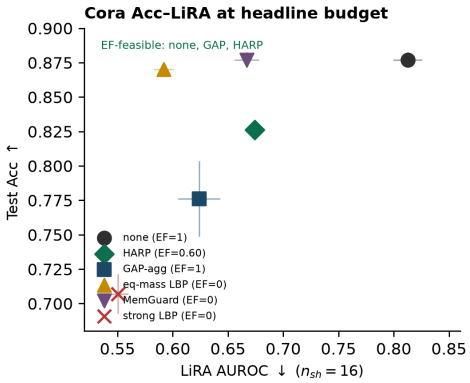

Fig. 4. Cora Acc–LiRA at the headline budget (n_{sh}=16, 5 seeds). ExactFrac-feasible: none, GAP-agg, HARP. Equal-mass LBP/MemGuard win unconstrained Acc–LiRA but ExactFrac=0.

GTD/MaskArmor report Acc/LiRA/ECE only (train-time; no Mass/Frac). GAP-agg is ExactFrac-feasible (train-only noise⇒ExactFrac=1) with finite ε, at Acc/ECE cost. HARP is the high-Acc ExactFrac option without training DP; HARP-full recovers strong-LBP Acc–LiRA under c=0. Selective masking keeps ExactFrac= 0.60 but raises LiRA.

d) HARP∘GAP: DP trainer as a component.: Proposition 2 says selective release is post-processing, so composing HARP on a GAP-trained model preserves GAP's (ε, δ). Measured (Cora, 5 seeds, σ=3): GAP alone Acc 0.770±0.030, LiRA 0.520; HARP∘GAP at Frac=0.40 keeps the same analytical $\varepsilon\approx$41, ExactFrac=0.60, LiRA 0.517, clean-slice conf AUROC 0.584, at Acc 0.649—composition costs Acc because GAP posteriors already carry training noise. The large ε is the Acc-preserving operating point of our GAP-agg reimplementation (σ=5 yields $\varepsilon\approx$19 at Acc 0.44); we do not market $\varepsilon\approx$41 as a strong privacy guarantee—only as a finite analytical budget that HARP does not inflate. Operators who need a formal ε *and* selective-release controls can compose; operators who need only ExactFrac keep plain HARP's higher Acc.

Slice ECE localizes the win on Cora (5 seeds): $\text{ECE}_{\text{clean}}$=0.018 vs. ECE_{prot}=0.081 (population 0.033); clean nodes match undefended calibration, protected nodes carry the noise penalty. The pattern is portable: clean-slice ECE beats undefended population ECE on Cora/Citeseer/Chameleon (0.020/0.083/0.273 vs. 0.053/0.140/0.290), while equal-mass LBP perturbs every response even when its population ECE looks competitive (Citeseer 0.073). ΔAcc vs. strong LBP at Frac= 0.40 ranges from +0.040 (Actor) to +0.185 (Photo) across six graphs at matched telemetry ($\text{Mass}_{\text{HARP}}$=0.4 Mass_{LBP}).

e) Multi-dataset portability.: Table III checks that selective release remains ExactFrac-feasible across six graphs under a matched light audit (n_{sh}=4; headline Cora uses n_{sh}=16). The secondary comparator is published strong LBP (b=0.3), which collapses Acc under full-graph noise; HARP recovers 4–19 Acc points at Frac=0.40 while keeping ExactFrac=0.60.

TABLE III
HARP PORTABILITY GRID (GRAPHSAGE; CORA 5-SEED, OTHERS 3-SEED; n_{sh}=4). HEADLINE CORA AUDITS USE n_{sh}=16 (TABLE II). TPR = LIRA TPR@1% FPR.

Dataset	Defense	Acc	LiRA	TPR	Mass	Frac
	none	0.877	0.567	0.033	—	—
Cora	LBP b=0.3	0.707	0.517	0.019	812	1.00
	HARP	0.832	0.533	0.011	325	0.40
	none	0.744	0.591	0.042	—	—
Citeseer	LBP b=0.3	0.620	0.523	0.030	998	1.00
	HARP	0.716	0.548	0.014	399	0.40
	none	0.448	0.618	0.059	—	—
Chameleon	LBP b=0.3	0.391	0.539	0.038	683	1.00
	HARP	0.452	0.555	0.011	273	0.40
	none	0.335	0.610	0.037	—	—
Actor	LBP b=0.3	0.287	0.549	0.032	2280	1.00
	HARP	0.327	0.561	0.019	912	0.40
	none	0.876	0.505	0.012	—	—
PubMed	LBP b=0.3	0.768	0.499	0.010	5915	1.00
	HARP	0.839	0.502	0.012	2366	0.40
	none	0.802	0.500	—	—	—
Photo	LBP b=0.3	0.552	0.499	—	2295	1.00
	HARP	0.737	0.501	—	946	0.40

TABLE IV
HARP FRAC SWEEP ON CORA (5 SEEDS; σ_{strong}=0.30). ACC/MASS ARE BUDGET-INVARIANT; LIRA COLUMN IS n_{sh}=4 (SEE TABLE II/ V FOR n_{sh}=16).

Frac	Acc↑	LiRA↓	Mass↓
0.20	0.859 [0.855, 0.866]	0.537 [0.532, 0.543]	162
0.30	0.847 [0.843, 0.853]	0.538 [0.533, 0.543]	244
0.40	0.832 [0.826, 0.839]	0.533 [0.525, 0.541]	325
0.60	0.800 [0.793, 0.809]	0.519 [0.512, 0.526]	487
1.00	0.714 [0.699, 0.727]	0.511 [0.503, 0.519]	812

This is *not* an unconstrained Acc–LiRA win (equal-mass LBP remains preferred when c=0). Heterophilic rows (Chameleon, Actor) show the largest undefended LiRA and the clearest case for a Frac budget; PubMed/Photo are near-chance controls where operators should set Frac=0 unless ExactFrac itself is required.

f) Frac dial and Mass accounting.: Table IV sweeps Frac on Cora (5 seeds): Acc falls monotonically as Frac rises under fixed σ_{strong}—the systems Acc–Mass dial. LiRA in this sweep uses n_{sh}=4 (cheap dial diagnostics); the headline privacy comparison is Table II at n_{sh}=16, and CFS under that budget is Table V.

g) Constrained Frac Search (matched to the headline audit).: CFS is only meaningful when τ tracks the attack budget: τ=0.55 is appropriate under n_{sh}=4 but infeasible under headline n_{sh}=16 (locked HARP LiRA≈0.67). We treat τ as an operator-selected risk tolerance—not a universal privacy target—and run CFS at n_{sh}=16 with ExactFrac≥ 0.60 and study cap τ=0.70 on *validation* Acc/LiRA (Table V); locked Frac=0.40 remains the pre-declared release-only dial reported on the held-out audit. Among Frac∈ $\{0, 0.2, 0.3, 0.4\}$, every constructor finds a feasible point; mean selected Frac is 0.23–0.33 with Acc≈0.84—strictly better Acc than the locked Frac=0.40 dial while meeting τ. Tighter τ=0.68 pushes topology to Frac=0.40; looser τ=0.75 selects Frac=0.20. Under ExactFrac alone (no τ), CFS degenerates to Frac=0; the audit cap is what forces a nonzero spend. Locked Frac=0.40 remains the conservative fixed operating point.

TABLE V
CFS ON CORA AT n_{sh}=16 UNDER EXACTFRAC≥ 0.60 AND LIRA≤ τ=0.70 (3 SEEDS; MEAN SELECTED FRAC/ACC/LIRA). ALL ROWS FEASIBLE_AUDIT=1.

Constructor	Selected Frac	Acc	LiRA
topology (LTE)	0.33	0.838	0.683
ensemble	0.30	0.838	0.681
random	0.23	0.839	0.689

TABLE VI
SESSION BUDGET ON HARP CORA AT K=20 (5 SEEDS). B=1 FREEZES ACC NEAR ONE-SHOT; B=20 MATCHES UNCAPPED. SHARED GLOBAL B=1 ACROSS N IDENTITIES BLOCKS SYBIL ACC RECOVERY (RIGHT).

Policy	B	Acc	LiRA	N ids	Per-id B=1	Shared B=1
Budget reuse	1	0.832	0.540	1	0.825	0.825
Budget reuse	5	0.851	0.540	5	0.874	0.825
Budget reuse	20	0.885	0.545	20	0.876	0.825
Uncapped avg	—	0.886	0.545	—	—	—

h) Exact scores ≠ membership safety.: Clean nodes are well calibrated ($\mathrm{ECE}_{\mathrm{clean}}$=0.018), but they are not automatically hard to attack. Under locked HARP, unprotected-slice confidence AUROC is ≈0.64 vs. ≈0.51 population-wide (Table XI); the mitigation stack below addresses it. Aerni et al. [37] motivate such canary checks (Sec. VI, canaries).

i) Velocity: multi-query and session budgets.: Averaging K independent releases denoises Laplace and moves toward the undefended baseline (Cora 5-seed LiRA at K=1/5/20: HARP 0.536/0.546/0.551; n_{sh}=4). A session policy of at most B fresh draws per *authenticated principal*—reusing the last draw thereafter—caps this drift (Table VI). **Enforcement assumption:** server-side state bound to authenticated identities with rate limits. Sybil-capable attackers who mint many principals can bypass per-identity B; Table VI shows N=5 identities under per-id B=1 recover Acc≈0.874 (near uncapped), while a *shared global* B=1 pool keeps Acc at the one-shot floor (0.825). Deploy shared B (or authenticated single-principal sessions) when Sybil velocity is in scope; Frac and B remain complementary controls.

j) Constructors: a privacy effect, not just an interface.: The decisive constructor metric is *clean-slice* conf AUROC: ranking removes vulnerable members from the bit-exact slice (Table VII). At matched ExactFrac=0.60 (5-seed Cora), slice AUROC falls with ranking quality: random 0.669±0.015, topology 0.647±0.012, audit 0.616±0.016, ensemble 0.594±0.010. Paired tests vs. random: audit Δ=0.053 ($p<10^{-3}$), ensemble Δ=0.075 ($p<10^{-3}$); Acc is equal-or-better (ensemble 0.813 vs. random 0.811). At the headline budget (n_{sh}=16, Frac=0.40), ensemble also lowers population LiRA vs. topology (0.646 vs. 0.674)—constructors are primarily a slice dial, with a measurable population effect under strong audits. Hop k∈$\{0, 1, 2\}$ at matched Frac changes Acc by $\lesssim$0.01 (release-only); selectivity, not train-time alignment, carries ExactFrac.

TABLE VII
RELEASE-ONLY CONSTRUCTORS AT FRAC=0.40 (CORA, 5 SEEDS; MEAN±1.96 SE). SLICE = CLEAN-SLICE CONF AUROC. LIRA HERE n_{sh}=4; HEADLINE IS TABLE II.

Policy	Feas.	Acc	LiRA	Slice↓	EF
none	yes	0.877±0.005	0.567±0.006	—	1.00
GAP-agg (σ=3)	yes	0.776±0.024	0.518±0.008	—	1.00
LBP equal-mass	no	0.870±0.005	0.520±0.011	—	0.00
LBP b=0.3 / HARP-full	no	0.707 / 0.715	0.517 / 0.512	—	0.00
sel. audit	yes	0.820±0.010	0.526±0.009	0.616±0.016	0.60
sel. ensemble	yes	0.817±0.011	0.527±0.014	**0.594±0.010**	0.60
sel. random	yes	0.810±0.006	0.526±0.007	0.669±0.015	0.60
sel. entropy	yes	0.805±0.010	0.530±0.014	—	0.60
sel. topology (LTE)	yes	0.826±0.006	0.536±0.008	0.647±0.012	0.60

TABLE VIII
RAWEXACTFRAC VS REPLAYFRAC (CORA, 3 SEEDS). GLOBAL-CACHE AND SEEDED LBP RESTORE REPLAY/CACHE WITHOUT RAW POSTERIOR EQUALITY; HARP MEETS EXACTFRAC=c WITHOUT A NON-RAW LEDGER.

Policy	RawEF↑	Replay↑	Cross↑	Cache↑
none	1.00	1.00	1.00	0.87
LBP fresh	0.00	0.00	0.00	0.00
LBP sticky (B=1)	0.00	1.00⋆	0.00	0.62
LBP global cache	0.00	1.00	1.00	0.87
LBP seeded (node,epoch)	0.00	1.00	1.00	0.87
HARP (Frac 0.40)	0.60	0.60	0.60	0.77

⋆Within-client sticky replay; cross-client ExactFrac stays 0. Clean-slice threshold flicker ≈0.09 for fresh/sticky LBP and 0 for global/seeded/HARP.

TABLE IX
IN-PROCESS RESPONSE-CACHE MICROBENCHMARK (CORA; 20K ZIPF QUERIES; CONTENT KEYS ON RELEASED VECTORS). ABSOLUTE QPS/LATENCY ARE NOT END-TO-END API NUMBERS.

Workload	Policy	Hit	QPS	p50 ms	p95 ms
zipf	none	0.937	4.0×10^6	<0.001	<0.001
zipf	lbp_fresh	0.000	9.3×10^3	0.104	0.116
zipf	harp_fresh	0.606	6.2×10^5	<0.001	0.004
zipf	lbp_B1	0.937	1.5×10^5	<0.001	0.101
zipf	harp_B1	0.937	2.2×10^6	<0.001	<0.001

k) Why ExactFrac matters (RawExactFrac ≠ ReplayFrac).: Caches and ledgers can stabilize previously sampled noisy outputs [40], [41]; that is ReplayFrac, not ExactFrac. Table VIII (Cora; 3 seeds; 8 clients; 20k Zipf; shared LRU 1024) makes the separation concrete: fresh LBP has both fractions ≈0; per-client sticky LBP raises within-client ReplayFrac but not cross-client equality or ExactFrac; *global* first-draw caches and deterministic seeded LBP achieve ReplayFrac=1 and high cache hits with ExactFrac=0; HARP alone among noisy policies meets ExactFrac=0.60 without storing non-raw scores. Table X confirms clean-slice bit-stability; Table IX is an *in-process response-cache microbenchmark* (not end-to-end networked API throughput; absolute QPS/latency are mechanism artifacts on Cora).

l) Clean-slice residual risk (first-class) and the mitigation stack.: Unprotected nodes are *not* claimed safe (Table XI): clean-slice conf AUROC rises to 0.63–0.67 while population LiRA stays ≈0.53. HARP attacks this from three composable directions (all at ExactFrac=0.60, 5 seeds): (i) *constructor*: ensemble ranking removes vulnerable members from the exact slice, 0.669 → 0.594 (Table VII); (ii) *deterministic confidence smoothing (DCS)*: released rows on the clean slice with $p_{\max}>\theta$ are flattened by a fixed temperature (θ=0.9, T=3)—a deterministic, argmax-preserving post-map, so re-queries stay bitwise stable, caches and audit replay still work, and routing decisions are unchanged; slice AUROC falls 0.650 → 0.618 under topology ranking and 0.591 → 0.576 on top of ensemble, with *zero* Acc change; (iii) *slice-aware CFS*: adds a slice-AUROC cap to the search. DCS trades raw-score fidelity on smoothed rows for slice privacy while preserving ReplayFrac and threshold stability; when ExactFrac (raw $\hat{p}_v$) is contractual, skip DCS. The stacked floor (≈0.575) is structural at c=0.60: a 60% deterministic slice must retain some membership signal. Lower c, or use GAP-agg / HARP∘GAP, if tighter slice control is required.

TABLE X
MEASURED (NOT DEFINITIONAL) REPLAY BEHAVIOR ACROSS INDEPENDENT RE-QUERIES (CORA, 3 SEEDS): BITWISE-EQUAL FRACTION, TOP-1 LABEL FLIPS, AND THRESHOLD ($p_{\mathrm{top}}>0.5$) ROUTE FLIPS. HARP'S FLIPS ARE CONFINED TO THE PROTECTED MINORITY; THE CLEAN 60% IS BIT-STABLE WITH ZERO FLICKER.

Policy	Bit-equal re-query	Top-1 flip	Thresh. flip
none	1.000	0.000	0.000
GAP-agg	1.000	0.000	0.000
LBP eq-mass	0.000	0.027	0.101
HARP (Frac 0.40)	0.600	0.146⋆	0.181⋆

⋆Entirely within the protected 40% (σ=0.30); clean-slice flip rate is exactly 0. LBP spreads lower-amplitude flicker over *all* nodes, so no response is replay-stable.

TABLE XI
LTE-AWARE ADAPTIVE QUERIES ON CORA HARP (5 SEEDS; CONF AUROC IS ATTACK-BUDGET FREE; LIRA@0.4 USES n_{sh}=4).

Query set	Conf@0.4	Conf@0.6	LiRA@0.4
Population	0.518	0.473	0.533
Unprotected (clean)	0.645	0.633	0.542
Boundary (clean)	0.634	0.628	0.538
Top-decile LTE	0.641	0.511	0.529

m) Inductive edge exclusion.: Dropping test-incident edges (Cora, 5 seeds) raises undefended LiRA to 0.818; HARP recovers Acc 0.476 → 0.601 vs. strong LBP at Frac=0.40.

C. When to spend: volume × structure

Table XII(a): undefended LiRA concentrates in small/mid heterophilic cells; Photo/PubMed/ogbn-arxiv sit near chance under matched shadows—audit first, then set Frac. Table XII(b) reports the honest shadow-count story on Cora for none / LBP / HARP. Absolute LiRA rises with n_{shadows} for every defense [4]; Acc is invariant. HARP does *not* beat strong LBP on LiRA at large shadow counts (Cora n_{sh}=64: LBP 0.561 vs. HARP 0.677)—expected when ExactFrac= 0 is allowed. What HARP keeps is (i) strictly lower LiRA than undefended and (ii) Acc recovery vs. strong LBP (ΔAcc +0.12, invariant in n_{sh}). TPR@1% FPR follows the same pattern.

TABLE XII
AUDIT CONTROLS (GRAPHSAGE; 3 SEEDS). ⋆HETEROPHILIC. (B) SHADOW SCALING (NOT HARP-ONLY). ACC INVARIANT IN (B): NONE 0.879, LBP 0.716, HARP 0.836.

(a) Undef. LiRA (n_{sh}=4)				(b) Cora LiRA / TPR@1%			
Dataset	**n**	**Acc**	**LiRA**	$\mathbf{n_{sh}}$	**none**	**LBP**	**HARP**
Chameleon⋆	2277	0.449	0.623	4	0.568/0.033	0.517/0.022	0.534/0.014
Cora	2708	0.879	0.568	16	0.808/0.201	0.555/0.008	0.670/0.057
Actor⋆	7600	0.335	0.610	64	0.847/0.265	0.561/0.008	0.677/0.075
Photo	7650	0.825	0.500				
PubMed	19717	0.876	0.505				
ogbn-arxiv	169343	0.666	0.503				

TABLE XIII
OGBN-PRODUCTS: STANDARD BFS LADDER VS. CANARY-STRESSED 15K (n_{sh}=4, 3 SEEDS). CAN. = CANARY LIRA.

Setting	**Acc**	**LiRA**	**Can.**	**EF**
Full load none (2.45M)	0.695	—	—	1.00
Std. 15k none/LBP/HARP	0.876/0.343/0.693	0.528/0.501/0.513	—	1/0/0.60
Canary none	0.861	0.593	0.997	1.00
Canary LBP b=0.3	0.379	0.581	0.805	0.00
Canary HARP	0.685	0.592	0.891	0.60

D. Canaries and large-graph volume

Cora canary checks: HARP suppresses planted/top-decile confidence cues relative to none/LBP (top-decile conf AUROC $0.551/0.548/0.266$ for none/LBP/HARP; planted $0.527/0.543/0.423$); population LiRA remains primary. Table XIV: on ogbn-arxiv (∼169k; NeighborLoader; gate off, warmup 3) [28], strong LBP drops Acc to 0.179 $[0.175, 0.184]$. HARP recovers 0.599 $[0.597, 0.601]$ (paired ΔAcc $+0.420$ $[0.414, 0.425]$; 3 seeds) at Frac=0.40. Undefended LiRA stays near chance at $n_{\text{sh}}\in\{2, 4\}$ and also at n_{sh}=16 (LiRA 0.510; Acc 0.666)—so the Acc recovery is a serving result under an audit-null regime, not under-powered shadows. On ogbn-products (∼2.4M nodes) [28], Table XIII reports a serving probe plus BFS LiRA at $n\in\{15\text{k}, 40\text{k}\}$ and $n_{\text{sh}}\in\{2, 4, 8\}$. Full-graph products LiRA OOMs on this host; we do *not* claim a million-node privacy audit. Across standard-recipe probes LiRA stays near chance while HARP recovers Acc vs. strong LBP (e.g., 15k/n_{sh}=8: HARP Acc 0.693 vs. LBP 0.343).

a) Canary-stressed probe (above-chance signal at scale).: Standard products/arxiv recipes sit near chance, so Acc recovery alone is a weak privacy claim. We plant 300 unique-feature canaries on the 15k BFS probe (half in train / half held out; n_{sh}=4; 3 seeds): undefended canary-LiRA is ≈1.00 while population LiRA is 0.593 (Table XIII). HARP cuts canary-LiRA to 0.891 (still high) at Acc 0.685 with ExactFrac=0.60; strong LBP reaches 0.805 only by collapsing Acc to 0.379 at ExactFrac=0. Population LiRA barely moves (≈0.59) under either release defense—defense-aware shadows recalibrate—so the scale takeaway is Acc-preserving ExactFrac under a hard canary stress, not a population-LiRA win at 15k. We do *not* claim a full 2.4M-node privacy audit.

If audit LiRA≈0.5, prefer Frac=0; if LiRA≫ 0.5 or ExactFrac≥ c is required, set Frac on val Acc.

b) Operator workflow.: Audit with defense-aware LiRA ($n_{\text{sh}}\geq$16 when affordable); if LiRA≈0.5 set Frac=0; else CFS on val Acc under (c, τ) (Table IV); prefer ensemble constructors + optional DCS for slice risk; enforce shared B=1 under Sybil velocity; use HARP∘GAP when a legal ε is required.

TABLE XIV
OGBN-ARXIV VOLUME (3 SEEDS). $\text{LiRA}_2/_4$ ARE DEFENSE-AWARE; UNDEFENDED LIRA AT n_{sh}=16 IS 0.510 (NEAR-CHANCE CREDIBILITY CHECK).

Def.	**Acc**	$\mathbf{LiRA_2}$	$\mathbf{LiRA_4}$	**Mass**
none	0.666	0.502	0.503	—
LBP	0.179	0.498	0.501	50 803
HARP	0.599	0.514	0.505	20 322

VII. DISCUSSION

HARP is a Big-Data score-serving mechanism for ExactFrac≥ c>0—not a claim to beat uniform noise when c=0, and not an equation of ReplayFrac with raw-posterior equality. The venue fit is prediction-API infrastructure under Volume/Velocity/Veracity: selective release, cache/replay accounting, and audits on high-volume graphs—membership privacy enters as an operator dial on that serving stack. ExactFrac formalizes a raw-score SLA that generic response caches do not automatically provide [40], [41]; c=0.60 is a study default, and sticky/global/seeded noise are not substitutes (Table VIII). GAP-agg already meets ExactFrac=1 with finite (often large) ε; HARP's complementary claim is high-Acc ExactFrac without training DP, plus constructors/CFS/slice controls (Prop. 2). **Limits:** (i) ExactFrac>0 ⇒ infinite global ε on released scores; (ii) clean-slice AUROC stays above chance at c=0.60—structural; (iii) CFS τ is an operator dial that must track the LiRA budget; (iv) products privacy is a BFS/canary ladder; (v) GAP-agg/MemGuard are reimplementations; (vi) serving benches are in-process microbenchmarks, not production traces. Future: learned constructors, distributed full-graph LiRA, production API traces.

VIII. CONCLUSION

HARP equips GNN prediction APIs with ExactFrac-constrained score serving: selective release meets raw-posterior c>0 where fresh and stateful-noisy uniform policies cannot, constructors cut clean-slice attack AUROC, CFS configures Frac under operator (c, τ), and HARP∘GAP preserves training DP for public-metadata constructors. Headline evidence uses n_{sh}=16; residual clean-slice risk and scoped large-graph Acc/serving probes are reported honestly. Use training-time DP (alone or under HARP) when a legal global ε is required.

a) Reproducibility.: Code (`defenses/harp.py`, `run_nsh16_headline.py`, `run_cfs_nsh16.py`, `run_exactfrac_sla_evidence.py`, `run_products_canary_leakage.py`, `run_bulletproof.py`), locked configs, and frozen CSVs: https://github.com/krishharma/privacy-gnn. Headline LiRA n_{shadows}=16; CFS uses τ=0.70 on validation; RawExactFrac SLA evidence in Table VIII.

REFERENCES

[1] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Security Privacy (S&P)*, 2017, pp. 3–18.

[2] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Proc. NDSS*, 2019.

[3] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE CSF*, 2018, pp. 268–282.

[4] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in *Proc. IEEE S&P*, 2022, pp. 1897–1914.

[5] I. E. Olatunji, W. Nejdl, and M. Khosla, "Membership inference attack on graph neural networks," arXiv:2101.06570, 2021.

[6] J. Jia, A. Salem, M. Backes, Y. Zhang, and N. Z. Gong, "MemGuard: Defending against black-box membership inference attacks via adversarial examples," in *Proc. ACM CCS*, 2019, pp. 259–274.

[7] Y. Wu, D. Li, B. Chen, and L. Shen, "Adapting membership inference attacks to GNN for graph classification," in *Proc. IEEE ICDM*, 2021, pp. 1429–1434.

[8] Y. Zhang, Y. Zhao, Z. Li, X. Cheng, Y. Wang, O. Kotevska, P. S. Yu, and T. Derr, "A survey on privacy in graph neural networks: Attacks, preservation, and applications," arXiv:2308.16375, 2023.

[9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[10] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[11] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. ICLR*, 2020.

[12] C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot, "Label-only membership inference attacks," in *Proc. ICML*, 2021, pp. 1964–1974.

[13] S. Sajadmanesh et al., "GAP: Differentially private graph neural networks with aggregation perturbation," in *Proc. USENIX Security*, 2023, pp. 3229–3246.

[14] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM CCS*, 2016, pp. 308–318.

[15] M. Nasr, R. Shokri, and A. Houmansadr, "Machine learning with membership privacy using adversarial regularization," in *Proc. ACM CCS*, 2018, pp. 634–646.

[16] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning," in *Proc. IEEE S&P*, 2019, pp. 739–753.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[18] J. Zhu et al., "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. NeurIPS*, 2020.

[19] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, "Geom-GCN: Geometric graph convolutional networks," in *Proc. ICLR*, 2020.

[20] D. Lim, F. Hohne, X. Li, S. L. Huang, V. Gupta, O. Bhalerao, and S. N. Lim, "Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods," in *Proc. NeurIPS*, 2021.

[21] P. Niu, C. Pan, S. Chen, and O. Milenković, "Graph Transductive Defense: a Two-Stage Defense for Graph Membership Inference Attacks," arXiv:2406.07917, 2024.

[22] C. Chen, X. Zhang, H. Qiu, J. Lou, Z. Liu, and X. Chen, "MaskArmor: Confidence masking-based defense mechanism for GNN against MIA," *Information Sciences*, vol. 669, 2024.

[23] A. Jnaini and M. Koulali, "Can diffusion-based posterior smoothing secure GNNs from membership inference attacks?" in *Proc. AICCSA*, 2025.

[24] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *Proc. WSDM*, 2020, pp. 76–84.

[25] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information," in *Proc. ACM CCS*, 2015, pp. 1322–1333.

[26] F. Tramèr et al., "Stealing machine learning models via prediction APIs," in *Proc. USENIX Security*, 2016, pp. 601–618.

[27] C. Dwork, "Differential privacy," in *Proc. ICALP*, 2006, pp. 1–12.

[28] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. NeurIPS*, 2020.

[29] P. Veličković et al., "Graph attention networks," in *Proc. ICLR*, 2018.

[30] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. ICLR*, 2019.

[31] I. Mironov, "Rényi differential privacy," in *Proc. IEEE CSF*, 2017, pp. 263–275.

[32] B. Jayaraman and D. Evans, "Evaluating differentially private machine learning in practice," in *Proc. USENIX Security*, 2019, pp. 1895–1912.

[33] C. Song, T. Ristenpart, and V. Shmatikov, "Machine learning models that remember too much," in *Proc. ACM CCS*, 2017, pp. 587–601.

[34] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. ICML*, 2005, pp. 625–632.

[35] H. Yuan, J. Xu, C. Wang, Z. Yang, C. Wang, K. Yin, and Y. Yang, "Unveiling privacy vulnerabilities: Investigating the role of structure in graph data," in *Proc. ACM KDD*, 2024, pp. 4059–4070.

[36] Y. Qi, X. Lin, and J. Wu, "Differentially private graph neural network with importance-grained noise adaption," arXiv:2308.04943, 2023.

[37] M. Aerni, J. Zhang, and F. Tramèr, "Evaluations of machine learning privacy defenses are misleading," in *Proc. ACM CCS*, 2024.

[38] X. He, R. Wen, Y. Wu, M. Backes, Y. Shen, and Y. Zhang, "Node-level membership inference attacks against graph neural networks," arXiv:2102.05429, 2021.

[39] M. Khosla, "Impact of graph structure on membership-inference risk for graph neural networks," *Proc. Priv. Enhancing Technol. (PoPETs)*, 2026.

[40] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *Proc. USENIX NSDI*, 2017, pp. 613–627.

[41] NVIDIA Corporation, "Triton Inference Server: Response cache," User Guide, 2024. [Online]. Available: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/response_cache.html